SCHOOL OF COMPUTING & INFORMATICS — PRACTICAL SYSTEMS ASSESSMENT

DELIMITED FLAT-FILE CATALOG MANAGEMENT SYSTEM

Coursework: **Python Systems Programming** | Duration: **2 Hours** | Marks: **40** | Focus: **Text File Handling**

1. Case Overview & Flat-File Storage Protocol

A municipal library requires a lightweight cataloging utility capable of tracking book accessions, genre categories, unit prices, and circulation stock. Unlike relational databases, this system uses a custom **pipe-delimited flat text file** (books.txt) for persistence. Students must demonstrate low-level file stream processing using Python's built-in file primitives (open, readline, write).

Delimited Record Protocol Specification (books.txt):
Format: <id>|<title>|<author>|<genre>|<price>|<copies>\n
Example raw file lines:
1|Python Programming|John Zelle|Technical|650.00|15
2|Clean Code|Robert Martin|Technical|950.00|8
3|The Great Gatsby|F. Scott Fitzgerald|Fiction|350.00|20

---

2. Recommended Modular Function Prototypes

To enforce clean software engineering principles, your solution should structure its business logic around the following modular function signatures:

```
def add_book_entry(catalog: list[dict], next_id: int) -> int:
    """Prompts user for book details, appends new dict, returns updated ID counter."""

def render_catalog(catalog: list[dict]) -> None:
    """Displays formatted tabular catalog or single-record card when count == 1."""

def query_books(catalog: list[dict], search_term: str) -> list[dict]:
    """Returns filtered list matching ID or case-insensitive title/author substring."""

def modify_book_details(catalog: list[dict], book_id: int) -> bool:
    """Updates price and copies for the specified book ID; returns success status."""

def sync_catalog_to_file(filepath: str, catalog: list[dict]) -> None:
    """Serializes each book dictionary into pipe-delimited strings in write mode."""

def load_catalog_from_file(filepath: str) -> list[dict]:
    """Parses books.txt line-by-line using split('|') and reconstructs dictionary list."""
```

Sample Catalog Seed Records:

ID	Book Title	Author Name	Genre	Price	Copies
1	Python Programming	John Zelle	Technical	650.00	15
2	Clean Code	Robert Martin	Technical	950.00	8
3	The Great Gatsby	F. Scott Fitzgerald	Fiction	350.00	20
4	Sapiens	Yuval Noah Harari	History	550.00	12
5	Cosmos	Carl Sagan	Science	480.00	6

3. Functional Requirements & CLI Controller

Implement an interactive menu loop with the following numbered options:
1. Add Book | 2. View Catalog | 3. Search Books | 4. Update Details | 5. Delete Book | 6. Save to File | 7. Load from File | 8. Exit

A. Catalog Registration & Display

- Prompt for title, author, genre, price, and copies. Auto-assign incremental ID starting from 1.
- Tabular catalog renderer must print cleanly aligned columns with header borders (refer to main.py). When only 1 book exists, render a detailed key-value inspection card.

B. Querying & Inventory Modification

- Search must accept either a numeric ID OR a string query matched against Title or Author (case-insensitive).
- Update function must ask for Book ID, allow updating unit price and available stock copies.
- Delete routine must prompt for explicit confirmation (y/n) before removing the record dictionary.

C. Flat-File Persistence Engine (books.txt)

- **Save Operation:** Open books.txt in write mode ("w") using a with context manager. Format and write each dictionary as a pipe-delimited line.
- **Load Operation:** Open books.txt in read mode ("r"). Iterate lines, apply .strip().split("|"), convert price to float and copies/id to int, and reconstruct the dictionary list.

---

4. Defensive Programming & Integrity Directives

- Strings (title, author, genre) must not be empty after stripping whitespace.
- Price must be **> 0.0** and Copies must be an integer ≥ **0**.
- Trap numerical casting errors with try-except ValueError.
- Handle missing books.txt on initial load using try-except FileNotFoundError.

---

5. Evaluation Matrix by Architectural Layer (40 Marks)

Layer 1	**Core Catalog CRUD Logic (Add, Render, Query, Update, Delete)**	**16**
Layer 2	Flat-File Text Protocol Serialization & Line Stream Parsing	10
Layer 3	Modular Function Architecture & CLI Menu Controller	8
Layer 4	Defensive Type Safety, Input Constraints & Exception Handling	6
Total	**Overall Examination Weightage**	**40**